

PropertyPilot — Cloud Provider Justification

Rubric mapping: Task 4.A.1 **Author:** Michael Riehm **Decision date:** June 2026 **Decision:** Deploy to **Render** across three managed services (PostgreSQL, Docker web service, static site).

1. Summary of the Decision

PropertyPilot's production environment runs on Render. The application is split across three Render services: a managed PostgreSQL instance, a Docker-built web service for the Express API, and a static site for the Vite-built React frontend. All three are deployed from the same GitLab repository and the same `Working` branch, with automatic redeploys on every push.

This document explains why Render was chosen over the realistic alternatives, and what the trade-offs are.

2. Evaluation Criteria

The capstone has a single developer, a fixed delivery window, and no operating budget. The evaluation criteria reflect that reality:

Criterion	What it means here
Cost	Must be free or near-free at the demo scale (one user, low traffic).
Docker support	Backend ships as a multi-stage Docker image — the platform must build and run Dockerfiles directly.
Managed Postgres	Self-hosting Postgres on a VM and managing backups is out of scope for a one-person capstone.
Source-control integration	GitLab webhook → automatic redeploy on push, with no separate CI/CD setup.
HTTPS by default	TLS at the edge with zero configuration.
Time to first deploy	Hours, not days. The rubric is graded on the application, not on infrastructure plumbing.
Predictable production parity	What runs in production should match what runs locally as closely as possible.

3. Why Render

3.1 Free tier with real Docker support

Render's free tier covers all three service types PropertyPilot needs — a PostgreSQL database, a Docker web service, and a static site — at zero cost up to the demo's traffic ceiling. The free Docker web service builds from a Dockerfile in source control with no separate registry step, which means the same `backend/Dockerfile` that powers `docker compose up` locally also runs in production unchanged.

3.2 Managed PostgreSQL

The Postgres add-on is fully managed — Render handles backups, version upgrades, and connection pooling. Provisioning is one click, and the connection string is injected into the backend service's environment variables. Because all three services live in the same Render region (Ohio), the backend talks to Postgres over Render's internal network using the "Internal Database URL", which is faster than an external connection and incurs no egress.

3.3 GitLab → Render webhook with zero CI/CD scaffolding

Render natively integrates with GitLab. After authorizing the connection once, every push to the `Working` branch triggers an auto-deploy on both the backend and the frontend services. There is no separate GitLab CI pipeline to write, no Docker registry to push to, and no deployment scripts to maintain. The git history is the deployment history.

3.4 HTTPS at the edge by default

Render terminates TLS at the load balancer for both the static site and the web service. Custom domains get automatic Let's Encrypt certificates. The Express app sees plain HTTP from the proxy but is correctly told the original protocol via `X-Forwarded-Proto`, which `app.set('trust proxy', 'loopback')` handles. No certificate management, no nginx reverse-proxy to maintain in production.

3.5 Production parity with the local stack

Locally, PropertyPilot runs on `docker compose up` with the same three components: Postgres in a container, the backend in a container, and an nginx static-site container that reverse-proxies `/api/*` to the backend. On Render, those three roles map directly to the three Render service types. The architecture diagram in [design-document.md](#) doesn't change between local and production — only the hosting overlay does. This is the single most valuable property of the choice: a bug that reproduces locally is almost certainly the same bug in production.

3.6 Simple operational story for a one-person team

Day-to-day operations are: push to git, watch the deploy log, hit the live URL. There is no Kubernetes cluster to upgrade, no autoscaling group to tune, no IAM policy to author. This is appropriate for the capstone's scope. If PropertyPilot ever needed to scale beyond a single instance per service, the same Dockerfiles would deploy onto ECS, Fly Machines, or a Kubernetes cluster with minimal modification — Render didn't paint the architecture into a corner.

4. Alternatives Evaluated

4.1 AWS (ECS Fargate + RDS PostgreSQL + S3/CloudFront)

Why considered. AWS is the industry default and the most powerful option of the three. ECS Fargate runs containers without managing nodes, RDS provides managed Postgres, and S3 + CloudFront serves static frontends globally.

Why not chosen.

- **Setup cost in time.** A working ECS Fargate + RDS + ALB + CloudFront stack requires at least one IAM role, a VPC with two subnets across availability zones, a security group, a target group, an Application Load Balancer, a CloudFront distribution, and an ECR repository to push Docker images to. The capstone has a hard delivery window and that time is better spent on the application.
- **Free tier doesn't cover RDS Postgres well.** The RDS free tier is twelve months and the `db.t3.micro` instance pushes against the rubric's "single landlord" scale assumptions. Beyond twelve months — past the WGU grading window — the cost is real.
- **Docker image registry overhead.** ECS pulls images from ECR, which requires a separate push step in a CI pipeline. Render builds the image inline from the Dockerfile in source control; AWS does not.
- **Disproportionate complexity.** PropertyPilot is a single landlord's app. The operational machinery of AWS is designed for organizations with multiple teams, environments, and compliance constraints. None of that

applies here.

AWS would be the right choice for a multi-region, multi-tenant SaaS product. PropertyPilot is neither.

4.2 Fly.io (Fly Machines + Fly Postgres)

Why considered. Fly is the closest direct competitor to Render. It builds Docker images from source, runs them as lightweight VMs, and has a managed Postgres offering. Pricing is comparable.

Why not chosen.

- **Less polished UX for a first-time deployer.** Fly's deployment story is CLI-first (`fly launch` , `fly deploy`). Render's is dashboard-first with the same CLI option. For a capstone where the deployment process gets documented for a grader, the dashboard story is easier to screenshot and narrate in the Panopto video.
- **Postgres is "managed" with caveats.** Fly Postgres is a managed image, not a fully managed service in the sense Render's is. Snapshots and backups exist but require more hands-on configuration. Render's Postgres is genuinely managed end-to-end.
- **No real disadvantage relative to Render** — Fly was a serious finalist. The decision came down to "the dashboard makes documentation easier."

Fly would have worked. It would not have worked materially better.

4.3 Vercel + Supabase (split stack)

Why considered. Vercel is the gold standard for hosting Vite/Next/React frontends, and Supabase provides managed Postgres with a generous free tier. The two together could host PropertyPilot if the backend were rewritten as serverless functions or split entirely.

Why not chosen.

- **Backend doesn't fit Vercel's model.** PropertyPilot's backend is a stateful long-running Express server with custom middleware (helmet, CORS, rate limit) and an init-time JWT-secret check. Vercel deploys serverless functions per route; refactoring the Express app into per-route handlers would have been a meaningful rewrite for no functional gain.
- **Split stack means split deploys.** A Vercel-hosted frontend talking to a Supabase-hosted Postgres directly via the Supabase client would have bypassed the backend entirely — but that breaks the rubric's repository-pattern and OOP requirements, which live in the backend. Keeping the backend somewhere else (Render, Fly, Railway) means juggling three providers instead of one.
- **No single deploy story.** Vercel deploys the frontend, Supabase manages the DB, and a third service runs the backend. Three dashboards, three sets of environment variables, three places to look when something breaks. The capstone benefits from a single pane of glass.

Vercel + Supabase is excellent for a serverless-first product. PropertyPilot is server-first.

4.4 Self-hosted (DigitalOcean Droplet or AWS EC2 + nginx)

Briefly considered for the cost ceiling. Ruled out because self-hosting Postgres on a single VM means owning backups, monitoring, OS patching, and TLS renewal — work that is out of scope for a one-person capstone and that adds zero rubric points.

5. Conclusion

Render wins on every criterion that matters for this project: it costs nothing at the demo scale, builds the existing Dockerfile without modification, ships managed Postgres in the same region, integrates with GitLab out of the box,

terminates TLS automatically, and is operationally trivial for a single developer. The alternatives are stronger on dimensions PropertyPilot doesn't care about (AWS for scale, Vercel for serverless, Fly for raw VM control) and weaker on the dimensions it does (time to first deploy and operational simplicity).

The trade-offs accepted are:

- **Free-tier cold starts** — Render's free Web Service spins down after fifteen minutes of inactivity, and the first request after that takes ~30 seconds. For a single landlord checking the app a few times a day, this is acceptable; for a real SaaS product, the next tier removes the spin-down.
- **Single region** — Ohio. There's no multi-region story on the free tier. PropertyPilot's user base is the developer's own learning project, so this is moot.
- **Lock-in is low.** Because the application is a stock Express + Postgres + Vite stack and all configuration is environment variables, moving to a different provider would require new Dockerfile build settings and a new Postgres URL — and nothing else.

Render is the right answer for PropertyPilot at this scope. If the project ever grew into a real product, the path forward (ECS Fargate, Fly Machines, or a managed Kubernetes cluster) would not require rewriting the application — only the deployment scripts.

6. References

Capability descriptions and pricing/feature claims about each provider above were drawn from the providers' own product and documentation pages, listed below.

- Amazon Web Services. (n.d.). *Amazon Elastic Container Service documentation*. <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html>
- Amazon Web Services. (n.d.). *Amazon RDS for PostgreSQL*. <https://aws.amazon.com/rds/postgresql/>
- Amazon Web Services. (n.d.). *AWS Free Tier*. <https://aws.amazon.com/free/>
- DigitalOcean. (n.d.). *DigitalOcean Droplets*. <https://www.digitalocean.com/products/droplets>
- Docker, Inc. (n.d.). *Docker documentation*. <https://docs.docker.com/>
- Fly.io. (n.d.). *Fly.io documentation*. <https://fly.io/docs/>
- Fly.io. (n.d.). *Fly Postgres*. <https://fly.io/docs/postgres/>
- Let's Encrypt. (n.d.). *Let's Encrypt — A nonprofit Certificate Authority*. <https://letsencrypt.org/>
- Render. (n.d.). *Render documentation*. <https://render.com/docs>
- Render. (n.d.). *Render pricing*. <https://render.com/pricing>
- Supabase. (n.d.). *Supabase documentation*. <https://supabase.com/docs>
- Vercel. (n.d.). *Vercel documentation*. <https://vercel.com/docs>